

Deep Q-Network (DQN) & DRQN Multi-Agent Hide & Seek: Architecture & Training Log

Alex-Nicolae Balint

June 5, 2026

Project Overview & Context

The Premise: This project is a custom Reinforcement Learning environment built from scratch, simulating a “Hide & Seek” scenario on a dynamic 2D grid. The environment challenges autonomous agents to learn spatial navigation, obstacle avoidance, and target interception entirely through trial and error.

The Agents & Mechanics:

- **The Seeker (Red Agent):** An AI agent equipped with a localized convolutional vision matrix (e.g., 5×5 grid view). Its goal is to locate and capture the Hider. It is penalized for hitting walls (-0.75) and wasting time (-0.05 per step), and heavily rewarded for catching the target ($+1.0$).
- **The Hider (Blue Agent):** A secondary agent acting as the target.
- **The Environment:** A Pygame-rendered grid featuring static maze geometry (bunkers, chokepoints) and interactive doors that can be toggled during runtime.

Technical Stack:

- **Python** core logic and object-oriented architecture.
 - **Keras/TensorFlow** for building and training the Deep Neural Networks.
 - **NumPy** for high-performance matrix operations and Bellman Equation processing.
 - **Pygame** for rendering the environment and handling real-time interactions.
 - **Matplotlib** for generating post-training performance metrics (Loss and Cumulative Reward).
-

Phase 1: The Initial Training & Performance Bottlenecks

- **Objective:** Implement the Bellman Equation to train the neural network via backpropagation.

- **The Problem:** The initial implementation caused the Pygame window to freeze. The program was attempting to train the neural network inside a `for` loop, calling the `predict()` function 128 times per frame, 30 frames per second.
- **The Solution (Vectorization):** Refactored the `train` function to process batches of data simultaneously. We extracted all states into NumPy arrays and reduced the network calls to just two `predict()` operations per batch.

Phase 2: Memory Persistence & Visual Metrics

- **Objective:** Allow the AI to save its learned weights and prove its learning mathematically.
- **Implementation:**
 - Integrated Keras `save_model` and `load_model` functions.
 - Shifted the paradigm to an “Episodic” structure with an `episode_total_reward` tracker.
 - Hooked into the `pygame.QUIT` event to dynamically generate timestamped Matplotlib dashboards (`performance_graphs.png`).

Phase 3: Behavioral Observations & “The Moving Target”

- **Observation:** After ~65 episodes, the graphs proved the AI was learning, but the Loss metrics were highly unstable with massive mathematical spikes.
- **Diagnosis:** The network was using its own constantly shifting weights to evaluate its future actions, creating mathematical instability.
- **The Solution (Target Network):** Split the brain into two separate neural networks: `active_brain` (The Student) and `target_brain` (The Teacher). Implemented `update_target_network()` to periodically hard-copy weights, providing stable Bellman targets.

Phase 4: Temporal Awareness (Frame Stacking)

- **Observation:** The Seeker struggled to track the Hider’s trajectory.
- **Diagnosis:** Convolutional Neural Networks (CNNs) possess amnesia; passing a single static frame provides no data regarding velocity or direction.
- **The Solution:** Implemented **Frame Stacking**. The agent’s memory state was upgraded from a 2D array to a 3D Tensor ($5 \times 5 \times 4$), stacking the 4 most recent chronological frames. This allowed the CNN’s filters to calculate motion vectors.

Phase 5: The Infinite Episode & Sparse Rewards

- **The Problem:** At 100% random exploration ($\epsilon = 1.0$), the agents mathematically rarely collided. Episodes dragged on indefinitely, causing the Hider to farm massive survival points and exploding the gradients.
- **The Solution (Curriculum Learning):**
 1. Introduced a strict `MAX_STEPS` timeout condition to cap episode length and teach the Seeker urgency.

2. Implemented “Phase 1 Training”: Temporarily disabled Hider’s brain to make it act as a stationary dummy. This allowed the Seeker’s neural network to successfully experience the +1.0 sparse reward and learn basic chase mechanics before initiating the Multi-Agent race.

Phase 6: Distribution Shift & Environment Geometry

- **The Problem:** If the AI trained on an empty grid, adding walls later would make the learned weights useless. Furthermore, static spawning allowed the Seeker to memorize a single route rather than learning generalized path finding.
- **The Solution:** Designed a permanent Level Architecture (including an L-barricade, pillars, and a central bunker). Implemented `randomize_spawns()` to dynamically place both agents on valid floor tiles at the start of every episode, forcing the AI to generalize its navigation across the entire maze.

Phase 7: The DRQN Architecture Upgrade

- **The Problem:** Because the Seeker’s vision is limited to a local 5×5 radius, it lacked a global compass. In empty hallways, moving left or right yielded the same empty -0.05 penalty. The agent became trapped in localized loops, “wiggling” between the same tiles.
- **The Solution (Stateful Memory):** Refused to implement artificial “Reward Shaping” in favor of a true architectural upgrade. Upgraded the agent from a standard DQN to a **Deep Recurrent Q-Network (DRQN)**.
- **Implementation:**
 - Restructured the input Tensor from a flat $5 \times 5 \times 4$ sandwich to a chronological 4D sequence: $(1, 4, 5, 5, 1)$.
 - Wrapped the Convolutional layers in Keras `TimeDistributed` layers to process each frame independently.
 - Injected a **Long Short-Term Memory (LSTM)** layer. The LSTM acts as a persistent memory cache across frames, granting the agent the temporal context required to recognize when it has been trapped in a local loop, overriding the CNN to pivot and explore new areas of the map.

Phase 8: State Space Refinement (The Vision Overlay)

- **The Problem:** Despite temporal architecture, the Seeker acted “blind”. Because the environment’s state array primarily tracked static geometry (0 for floors, 1 for walls), the Convolutional filters had no mathematical representation of the Hider to latch onto.
- **The Solution (Dynamic Coordinate Injection):** Modified the `observe()` function to calculate relative distances between the agents. If the target fell within the 5×5 local vision bounds, a unique scalar value (e.g., 9) was dynamically painted into the matrix. This defined a proper State Space, allowing the CNN to visually recognize and track the target.

Phase 9: Overcoming Sparse Rewards (Flashbulb Memory & Curiosity)

- **The Problem:** Positive rewards (+1.0) were mathematically drowned out by thousands of negative timeout steps, causing hesitation loops and preventing the weights of the network to update correctly.
- **Implementation:**
 1. **Curiosity:** Inspired by DeepMind’s approach, a local footprint tracker (`visited_tiles`) was added. The agent receives a micro-penalty (-0.05) for stepping on previously visited tiles within an episode, physically repelling it from wiggling in corners and forcing map exploration.
 2. **Flashbulb Memory (Lite Prioritized Experience Replay):** To balance the Replay Buffer dataset, the winning frame (capture) was artificially duplicated multiple times before being appended to memory. This amplified the positive gradient update during backpropagation, rapidly cementing the chase behavior.

Phase 10: Multi-Agent Reinforcement Learning (MARL) & Non-Stationary Targets

- **The Problem:** Transitioning to MARL woke up the Hider. The environment shifted from Stationary (static target) to Non-Stationary (active evasion). This caused an immediate **Q-Value Collapse** in the Seeker’s neural network. Its previously memorized routes failed, resulting in continuous timeouts. The Seeker became mathematically depressed (fearful of timeouts) and resorted to localized wiggling to minimize penalties.
- **The Solution (Exploration Forcing & Reward Inversion):**
 - **Reward Shaping:** Inverted the Bellman rewards for the Hider based on its assigned role variable (rewarded for survival, heavily penalized for capture).
 - **Epsilon Shock Therapy:** Refused to rely on mathematical heuristics (like algorithmic “scent” tracking or distance radars) to maintain pure organic learning. Instead, the Seeker’s exploration rate (ϵ) was artificially reset to high values ($0.40 - 0.50$). This chaotic exploration forced accidental collisions with the moving target, allowing the neural network to rebuild its Q-values and learn predictive pathing (cornering/trapping) through pure trial and error.

Training Results & Final Milestone

As of the current iteration, the Multi-Agent system has successfully demonstrated the classic “Arms Race” phenomenon inherent to MARL:

- The Seeker was successfully trained independently for **12,000 episodes**, developing highly effective hunting algorithms and cornering strategies.
- The adversarial framework (Seeker vs. Hider) was trained for a combined **4,000 episodes**.
- Log telemetry confirms that reliance on random actions ($\epsilon = 0.01$) has been successfully phased out. The agents now operate almost entirely on learned Convolutional and LSTM logic, showcasing predictive interceptions, evasive map sweeps, and dynamic pathfinding.